

Highlights

From a Production OTP Flood to a Layered Defence: Measured Limits of Risk-Scored SMS Verification Against Residential Flooders and Colluding Carriers

Naveed Ahmed

- An operator's account of an OTP flood, its first fix and the attack that returned
- Signature attacks contained, simulated completion within 5 points of no attack
- A human-like residential flooder was held by rationing; its cost is measured
- Pumping on few destination blocks: five sends per block plus those in flight
- Sequential tests and flat counters compared at matched legitimate traffic

From a Production OTP Flood to a Layered Defence: Measured Limits of Risk-Scored SMS Verification Against Residential Flooders and Colluding Carriers

Naveed Ahmed^{a,*}

^a*Affiliation to be completed before submission, Saudi Arabia*

Abstract

An endpoint that sends a one-time password by SMS to any number it is given will be made to send SMS at its operator's expense, to waste money (flooding) or to earn part of the termination fee (SMS pumping). Starting from such an incident, as its operator recalls it, we present a twelve-step pipeline that derives trust from app attestation, signed sessions and verification outcomes, and evaluate it in a seeded simulation whose offered traffic is fixed before the defence sees it, against six simpler designs and 23 attacker profiles, including colluding carriers. No production traffic was available; every figure depends on the stated calibration. Attacks with a signature in the request are contained while 75 to 80 % of simulated users complete (79 % in runs without attack). A residential attacker with farmed CAPTCHA scores is not separated at acceptable cost by any signal tested; an adaptive cap holds it to 236 of about 600 messages by refusing 54 % of first-time users, given a fast controller with history. Against pumping on three destination blocks, a sequential block test leaks 88 messages, as a closed-form estimate reproduces; a carrier that verifies 60 % of its codes, builds trust first or fakes receipts defeats it. At matched legitimate traffic, settings that hold that carrier to 152 messages or fewer raise 150 to 400 false verdicts a day at 65 % conversion, and a flat per-block counter holding it to 14 to 59 completes 2 to 17 % of registrations on busy blocks.

Keywords: SMS pumping, artificially inflated traffic, one-time password, rate limiting, risk scoring, residential proxies, sequential tests, fraud

*Corresponding author.

Email address: optimistbs@gmail.com (Naveed Ahmed)

1. Introduction

Most online services verify a phone number by sending it a code by SMS. Each message costs from a tenth of a cent to twenty cents, and the endpoint that sends it is reachable before any account exists. A flooder makes the service send thousands of messages to numbers it does not own; a pumper sends them to a range whose operator shares the termination fee with it, turning the service’s SMS budget into its income [1]. Twitter’s owner stated in 2023 that the company had been losing about 60 million dollars a year to such pumping [2], and Huh et al. found as many attack requests as genuine ones in three months of one large authentication service’s logs [3].

This paper starts from such an incident on a registration flow the author operated. The first fix, a hidden CAPTCHA, per-number and per-address limits, a country allow-list and a proxy block, stopped the first attack. The one that came back rotated residential addresses, sent the header that marked a request as coming from the iOS app, and stayed under every threshold. The second design is a twelve-step pipeline with a feedback loop: platform exemptions require app attestation, clients carry signed sessions, numbers are checked before a message is paid for, keys carry a reputation built from whether their codes were entered, a risk score chooses among five responses, and a hard budget bounds spend.

With no production logs available, we evaluate the design in simulation and ask which attacks it contains against six simpler designs, and at what cost in completed registrations (RQ1); which layer is responsible (RQ2); when outcome-based reputation separates an attacker from simulated users (RQ3); what rationing costs when it does not (RQ4); and what tests on the destination number block cost legitimate users under miscalibration, outage and deliberate poisoning (RQ5). The contributions are the incident as eight named gaps, the design and an open implementation, measurements of where reputation stops separating and what rationing depends on, a closed-form estimate of pumping leakage with a comparison of sequential tests and flat counters at matched legitimate traffic, and a collateral-damage study of destination-keyed reputation. Most answers narrow the design’s claims.

33 2. Background and related work

34 *Revenue-share fraud.* In international revenue share fraud a party advertises
35 a range of numbers, traffic is generated to them, and the termination fee
36 is split along the route [4]. Sahin and Francillon collected 3.14 million test
37 numbers from twelve such providers over four years and showed that the
38 number range is the fraudster’s managed asset, about 80 % of ranges being
39 shared across providers [5]. They also showed the danger of acting on ranges:
40 coarse blocking has made whole countries unreachable, and their range-based
41 detector had a 5.8 % false-positive rate against 0.26 % for a classifier on call
42 features.

43 *Detecting artificially inflated SMS.* Huh et al. trained gradient-boosted trees
44 on 9.4 million SMS authentication logs from nineteen countries, labelled by
45 a peak-traffic heuristic, and reached 89.6 % recall at 0.19 % false positives
46 on web traffic and 91.1 % at 0.11 % on native traffic [3]. About 73 % of web
47 attacks sent four or fewer requests per identity a day, so per-entity limits fail;
48 destination signals were the strongest, with about 92 % zero conversion for
49 attacks against 21 % for genuine traffic. We did not reproduce their detector,
50 which needs labelled traffic. Our question is narrower: for the feature family
51 they found strongest, outcomes per destination key, we make the detector
52 explicit and measure where it separates, what it costs legitimate users, how
53 its memory trades evasion against false verdicts, and how it compares with
54 a flat destination limiter at the same legitimate service.

55 *Defences in practice.* Twilio’s Fraud Guard blocks regions known for pump-
56 ing and destinations new to an account [6]; its SMS Pumping Risk Score
57 scores each number from carrier, traffic and conversion signals and advises
58 friction from 60 and refusal above 90 [7]. Graded action on the destination is
59 thus established practice; its model, memory and false positives are unpub-
60 lished, and they are what we measure for one explicit design. CAPTCHA
61 solving is a labour market at about a dollar per thousand [8], and the human
62 route stays open even where machine solvers fail [9]; client puzzles price a
63 request in computation [10], which we did not evaluate. Residential proxy ad-
64 dresses rotate within minutes and are rarely blacklisted [11], and carrier-grade
65 NAT makes address reputation on cellular networks “all but infeasible” [12].

66 *Identity and tests.* Absent a central authority, one entity can present many
67 identities [13]; a signed session gives continuity, not scarcity, of identity. Our

68 block tests are Wald’s sequential tests [14] run as Page’s cumulative sums [15]
69 with a bounded floor; time-uniform confidence sequences [16] and windowed
70 alternatives were not evaluated. Step 7 is a risk engine of the kind risk-
71 based authentication uses [17]; attacks on the code once sent are a separate
72 problem [18, 19].

73 3. The incident and the adapted attacker

74 This section is motivation, not evidence: no logs from the period exist
75 in our hands, no counts or bills are given, and permission to describe the
76 incident is being sought. The registration flow sent an SMS to any unregis-
77 tered number, and generated numbers were submitted repeatedly. The first
78 controls, v1, were reCAPTCHA v3 on web requests, one message per num-
79 ber per minute, five requests per address per minute, a country allow-list,
80 a body-length limit, a proxy block and per-platform caps. The abuse that
81 returned used thousands of residential addresses, carried the iOS header and
82 so skipped the CAPTCHA, and stayed under every limit. Table 1 lists the
83 eight gaps the review found.

Table 1: The v1 controls and the gaps the adapted attacker exposed.

Gap	v1 assumed	How it was bypassed
A	A platform header says which app sent the request	Sending the header reached every exemption
B	Five requests per address per minute bound an attacker	A fresh residential address per request
C	A number given is worth sending to	Random numbers cost as much as real ones
D	An allowed country is a safe destination	Premium ranges inside it cost far more
E	Per-key caps bound total spend	A wide attack under every cap
F	Check-then-increment is a rate limit	Two racing instances both passed
G	Bulk and test flags are internal	Neither needed stronger authentication
H	Each check says which check failed	Thresholds probed one at a time

84 *Threat model.* The attacker wants the service to send as many messages
85 as possible, or to send them to numbers that pay it. It buys residential
86 proxies at about 3 USD per gigabyte and human-like reCAPTCHA tokens at

87 about 0.3 cents, mints fingerprints for free, generates numbers, and knows the
88 thresholds. A pumper also controls destination ranges through a colluding
89 operator, may enter the codes delivered there, report deliveries as failed, build
90 verified history first, and buy challenge solutions. It cannot forge an app
91 attestation, change which ranges its partner terminates, or make a stranger
92 enter a code. We assume undelivered messages are still billed, which favours
93 the attacker.

94 4. Design

95 No message is paid for before every check passes, and cheap checks run
96 first. Trust comes from evidence the client cannot forge, an attestation, a
97 server-signed session and the fate of its earlier codes; such evidence can still
98 be manufactured, and Section 7.7 measures what that buys.

99 4.1. *The pipeline and the risk score*

100 A request passes the twelve steps of Table 2 until one refuses it. Step 0
101 derives the platform only from a valid App Attest or Play Integrity result,
102 so a forged header is ignored. Steps 2, 8 and 11 are atomic in Redis, and a
103 claim is released if a later step refuses. Every outcome after Step 0 except
104 the challenge returns one HTTP 200 body padded to 400 ms.

Table 2: The twelve steps and the v1 gap each targets. FB is the feedback loop.

Step	Check	Gap
0	Proxy block; platform from attestation, never from the header	A
1	Signed session bound to a fingerprint; 3 requests per session per 10 min	B
2	Atomic limits per address, /24 and ASN, lower for hosting ASNs	B, F
3–4	reCAPTCHA v3 threshold and origin check, web only	–
5	Number intelligence: cost class, sequential numbers, liveness; bypass needs a credential	C, D, G
6	Body length, excluded numbers; test bypass needs a credential	G
7	Risk score and tier: allow, delay, challenge, downgrade, block	H
8	Per-number backoff, 60 s doubling to 1 h, five a day	–
9	Per-source, platform and country caps with an adaptive multiplier	–
10	Operating mode: elevated above 80% of the hourly budget, emergency at 100%	E
11	Atomic hourly count and spend reservation; state written before sending; uniform response	E, H
FB	Verified, failed, undelivered per key; destination-block tests; denylist of fine keys	–

105 Step 7 computes

$$s = \sum_i w_i x_i + w_c \frac{\max(0, \theta - r_{\min})}{\theta} + w_s f + w_m i - w_v v_n - w_f v_f, \quad (1)$$

106 where the x_i are request signals (CAPTCHA score, hosting ASN, fingerprint
107 age, session requests, country mismatch, number class), $r_{\min}$ is the lowest
108 conversion ratio over the request’s keys (address, subnet, ASN, fingerprint,
109 country, prefix, destination block), $\theta = 0.3$, $w_c = 25$, f and i flag a flooded
110 key and a key whose codes are entered within seconds, and v_n , v_f mark a
111 number and fingerprint with verified history. Weights of 10 to 25 points are
112 hand-set and listed in the repository [20]. Tiers start at 20, 40, 60 and 80:
113 *allow* sends; *delay* sends after a doubling wait; *challenge* shows an interactive
114 challenge and credits -20 when solved; *downgrade* uses push or WhatsApp,
115 never SMS; *block* refuses. Two consequences matter later. The conversion
116 penalty attaches to keys legitimate users share, so alone it reaches only *delay*.
117 A datacenter address with a fresh fingerprint scores about 47, *challenge*, so
118 datacenter rotation is stopped only while the attacker declines challenges.

119 4.2. The feedback loop

120 A send resolves as *verified* when its code is entered, *failed* after five wrong
 121 attempts or two minutes past the delivery receipt, and *undelivered* without
 122 a receipt within sixty seconds or with a failure receipt; only the first two
 123 feed reputation. A repeated receipt changes nothing, a negative receipt after
 124 a positive one is logged as a conflict, a positive one after a failure reopens
 125 the send, and a late verification reclassifies the send, while a verdict already
 126 issued stands until it expires. Speed is clocked from the receipt. Fine keys under
 127 0.1 conversion over fifty sends are denylisted for a day, never a residential
 128 ASN.

129 4.3. Destination-block tests

130 Every key above can be rotated except the destination: a pumper is paid
 131 only on its partner’s numbers. We key reputation on the 8-digit block, 10 000
 132 numbers, and judge it with two log-likelihood-ratio statistics. The conversion
 133 statistic assumes legitimate users verify with $p_\ell = 0.8$ and a flooder with
 134 $p_a = 0.1$: a failure adds $\ln \frac{1-p_a}{1-p_\ell} = 1.50$, a verification $\ln \frac{p_a}{p_\ell} = -2.08$. The
 135 speed statistic assumes a machine enters a code within five seconds with
 136 $q_a = 0.9$ and a person with $q_\ell = 0.2$, because autofill APIs remove the
 137 keystroke [21, 22]; it adds 1.50 or -2.08 . A verdict is issued when either
 138 exceeds $h = \ln 1000 = 6.91$, and the statistic resets.

139 The floor is the detector’s memory: $S_t = \max(-ch, S_{t-1} + \text{increment})$,
 140 alarm at $S_t > h$. Unbounded c is Wald’s test, under which a hundred verified
 141 sends postpone a later flood’s detection until about 143 failures; $c = 0$ is
 142 Page’s cumulative sum. With $Y_t = S_t + ch$, any finite c is a zero-floor
 143 CUSUM with threshold $(1 + c)h$ and head start ch , so Section 7.5 treats
 144 thresholds and credits as one grid. The default is $c = 1$.

145 A first verdict makes the block’s first-time clients pass an interactive
 146 challenge for an hour; a second within the hour moves them off SMS, which
 147 refuses the 30 % of simulated users without another channel. Clients with
 148 verified history are exempt, and a 24-hour denylist is kept as a comparator.
 149 The tests pause for fifteen minutes on a prefix whose receipts, or whose
 150 returning clients’ verifications, collapse across ten blocks.

151 4.4. Capacity controls

152 Step 9 multiplies its caps by 0.25 to 1.5 according to a baseline job that
 153 compares each key with its expected rate, learned from the pipeline’s own
 154 hourly counters over the closed hours available; with none, the static cap

stands. The default worker runs it every minute, and clients with verified history are not rationed. Step 11’s reservation is a hard ceiling on SMS count and spend units, not on vendor or fallback costs.

5. An estimate of pumping leakage

Against the block tests, leakage depends on the blocks a pumper touches. With B blocks reached, rate λ , N requests and $k = \lceil h/1.50 \rceil = 5$ sends per block before a verdict,

$$L \approx \min(N, kB + \lambda \tau), \quad (2)$$

where τ is the time a send stays unresolved: about 2.5 minutes for a carrier that never verifies, seconds for one that verifies at once. It assumes empty history, no credit, solved challenges, exemptions or suspension, and is evaluated on the blocks each run touched: a consistency check of the per-block mechanism, not a forecast, and once $kB > N$ it only says everything leaks.

A carrier verifying a share σ of its codes with human-like delay gives the conversion statistic an expected increment of $\sigma(-2.08) + (1 - \sigma)(1.50)$, zero at $\sigma^* = 0.42$. Below it, detection is expected with a delay that grows near σ^* ; above it, the drift is away from the threshold and a verdict within a finite window depends on the floor and on runs of failures: delayed, not excluded. Each entered code is a verification on a pumper’s number that a registration flow would turn into an account; we count verifications, not accounts.

6. Implementation and evaluation method

Implementation. About 3 000 lines of Python implement the pipeline, feedback loop and adapters for reCAPTCHA, Play Integrity, App Attest, IP intelligence, number lookup, SMS and WhatsApp, on in-memory and Redis backends. App Attest assertions are verified against enrolled keys; enrolment itself is not implemented. Step 11 writes the audit record, counters and OTP entry before calling the sender, so a synchronous callback lands on registered state, and receipt, verification and block-test transitions are atomic read-modify-writes (WATCH/MULTI/EXEC on Redis). Of 273 tests, five run two pipeline instances on a real `redis-server` and establish, on that backend only, that the hourly ceiling holds under concurrent sends, a number is claimed once under sixteen concurrent requests, six concurrent callbacks for one message count once, concurrent block events issue exactly the expected

187 verdicts, and compare-and-set serialises eight writers; the rest use in-memory
 188 and fakeredis stores. No live vendor was called.

189 *Simulation.* The pipeline runs inside a simulated environment: scripted ven-
 190 dor fakes, a simulated clock, receipts and code entries as events, and a base-
 191 line job told the modelled legitimate rate (an oracle) except in Section 7.4.
 192 After a ten-minute warm-up (130 for the pumping study), each run has
 193 twenty minutes of attack over legitimate traffic at 20 requests per minute.
 194 Simulated users verify 80 % of codes after a lognormal delay (median 22s, 2s
 195 for the 20 % using autofill); 20 % reuse an earlier identity; 70 % have What-
 196 sApp; 90 % complete a challenge. Sessions pass the same CAPTCHA gate as
 197 the endpoint, and a challenged party retries with the same session and num-
 198 ber. Every exogenous quantity of a request, including a retry’s CAPTCHA
 199 score and which identity a returning request reuses, is drawn when it is cre-
 200 ated; a test asserts that the trace hash is identical across designs, so designs
 201 are paired on one trace. Requests are bound at creation to immutable first-
 202 time or returning cohorts, every run asserts that cohorts reconcile and that
 203 completed $\leq$ delivered $\leq$ dispatched $\leq$ offered, users are counted before the
 204 session gate, the clock advances whatever the pipeline decides, and runs drain
 205 with nothing pending.

206 Prices are list prices, such as 0.1422 USD per SMS to Saudi Arabia.
 207 Conversion, delays, autofill, returning share, fingerprint freshness, challenge
 208 completion and CAPTCHA score distributions are assumptions, each marked
 209 in the calibration file with the published figure it was set against where one
 210 exists (one provider reports 68 % conversion [23]); seeds bound sampling
 211 variability, not these choices. The main study uses thirty seeds per attacker,
 212 drawing pool size (500 to 50 000 addresses), rate (10 to 60 per minute) and
 213 CAPTCHA class per seed; sweeps, ablation, cadence, pumping and detector
 214 studies use ten, the baseline-job, dilution and 24-hour studies five, the outage
 215 study eight. Brackets give 95 % t -intervals. Runs either lift the source caps
 216 or apply them at three times the legitimate rate.

217 *Designs and metrics.* Feature flags give seven designs: **v1** (header-trusted
 218 platform, static caps), **v1-corrected** (platform from attestation), **budget-**
 219 **only** (sessions, static caps, hard budget), **block-limit-only** (sessions, five
 220 sends per block per day), **conversion-only** and **speed-only** (v2 minus one
 221 block test), and **v2**. The ablation removes one of ten flags; removing the
 222 circuit breaker removes Step 10 and the ceiling. We report messages leaked

223 in twenty minutes; seeds contained (leakage at most 5% of the attack rate
 224 to the end) with time to containment over contained seeds only; legitimate
 225 requests dispatched, delivered, completed, challenged and refused as shares
 226 of those offered, for all and for first-time clients; and verdict events, distinct
 227 blocks, events by stage, requests hit and those never completed. Table 3 lists
 228 the attackers.

Table 3: Attacker profiles. Residential ones use in-country ranges; “farmed” CAPTCHA scores follow the human distribution.

Profile	Description
single client	One address and session, random numbers, bot scores
datacenter	Hosting ASN abroad, fresh fingerprint per request
residential bot / farm	Fresh fingerprints; bot / farmed scores (farm also with aged fingerprints)
reused profile	One 48-hour-old browser profile
sequential / premium	Numbers walked upward / a premium-rate prefix
forged header	The farm sending the iOS header without attestation
listed-range pumper	Elevated-cost range; verifies all within 1 s, or 60 % after 30 s
verifying flooder	All standard prefixes, 60 % verified after 30 s
challenge solver	Datacenter rotation buying every challenge solution
low and slow	2 or 10 requests/min over 20 residential ASNs
trust builder	500 identity/number pairs verify for 10 min, then flood; also concentrated in three blocks
receipt faker	Concentrated pumper reporting every delivery failed
block poisoner	Floods the 20 blocks simulated users concentrate on
concentrated pumper	Three 10 000-number blocks; carrier never verifies, verifies within 1 s, verifies 60 % after 30 s, or buys challenge solutions

229 7. Results

230 7.1. RQ1: what is contained, and at what cost

Table 4: Leaked of about 600 requests in twenty minutes, thirty seeds, under v1, the flat block limit (BL) and v2; with caps on, also first-time users refused and all users completed. v1-corrected equals v1 except on the forged header (599 lifted; 562, 4 %, 77 % on); budget-only equals v1-corrected except on the single client (2) and premium (551; 545, 6 %, 75 %); conversion-only equals v2; speed-only differs only on the walk (76; 69, 4 %).

Attacker	lifted: leaked			caps on, v1			caps on, v2		
	v1	BL	v2	leak	refused	compl.	leak	refused	compl.
Single client	31	2	2	31	1 %	79 %	2	1 %	79 %
Datacenter	412	412	0	392	3 %	79 %	0	1 %	80 %
Premium	591	5	0	558	4 %	76 %	0	1 %	79 %
Sequential	591	5	70	558	4 %	76 %	63	3 %	77 %
Reused profile	599	599	124	562	4 %	77 %	94	7 %	75 %
Residential bot	210	210	210	210	1 %	79 %	154	15 %	69 %
Farmed CAPTCHA	599	599	599	562	4 %	77 %	236	54 %	42 %
Forged header	602	599	599	100	1 %	79 %	236	54 %	42 %

231 Datacenter rotation leaks nothing under v2, because every request lands
 232 in the challenge tier and this attacker declines challenges; premium pumping
 233 leaks nothing because Step 5 refuses the range; the session cap holds the
 234 single client to two (Table 4, Figure 1). The sequential walk leaks 70 [58,
 235 82] before its block’s verdict at minute 3.0, the reused profile 124 [110, 138]
 236 before its fingerprint’s history catches it, at minute 4.6 on average. Under
 237 these attacks 75 to 80 % of simulated users complete with caps on, against
 238 79 % [79.0, 79.6] in 24-hour runs without attack, and 1 % of first-time users
 239 are refused except under the sequential walk (3 % [3, 4]) and the reused profile
 240 (7 % [6, 9]), which tighten the adaptive cap before being caught. The cost is
 241 small, not zero.

242 Budget-only contains only the single client: a hard budget bounds spend
 243 but separates nothing. The flat limit of five sends per block per day stops the
 244 sequential walk and the premium pumper at five messages, better than v2’s
 245 63 to 70, at no measured cost here because few legitimate sends reach a block
 246 in twenty minutes; Section 7.5 compares the two where legitimate sends are
 247 dense. The forged header buys nothing under v2, where it is ignored; under
 248 v1 it is believed and puts the attacker in the app bucket, whose static cap
 249 holds it to 100. Attestation removes the attacker’s choice of bucket; it does
 250 not by itself reduce leakage.

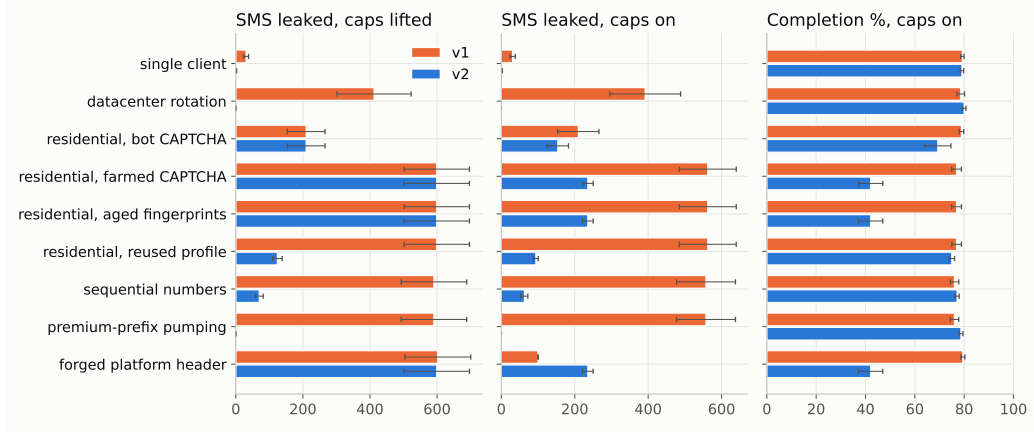


Figure 1: Messages leaked under v1 and v2, caps lifted and on, and legitimate completion with caps on; thirty seeds, 95 % t -intervals. Spread in leakage is mostly the per-seed attack size; rationing shows in completion.

251 The farmed-CAPTCHA attacker leaks 599 [501, 697] with caps lifted
 252 under every design. With caps on, v1’s static cap leaks 562 [485, 640] and
 253 refuses 4 % of first-time users; v2’s adaptive cap leaks 236 [221, 250] by
 254 refusing 54 % [47, 61] of first-time and 22 % of returning users, so 42 % [37,
 255 47] complete instead of 77 %. That gain is rationing.

256 7.2. RQ2: which layer does what

257 Each signature attack depends on one or two layers (Figure 2): the ses-
 258 sion cap for the single client (+28 [+15, +42] without it), the risk engine
 259 for datacenter rotation (+86 [+72, +100]), number intelligence for premium
 260 pumping (+76 [+52, +99]). The feedback loop stops the reused profile (+140
 261 [+120, +161], and delivery falls 40 points as the cap tightens on everyone)
 262 and, with the block key, the sequential walk (+152 [+137, +168] without the
 263 loop, +123 [+101, +145] without the key, and +174 [+164, +184] without
 264 the risk engine that acts on both). Only the adaptive cap matters against
 265 the farm (+313 [+181, +444], delivery 42 points higher). Removing the
 266 circuit breaker changes nothing, since no twenty-minute attack reaches the
 267 hourly budget. Of the ninety cells, 73 leave mean leakage unchanged; the
 268 supplementary results give intervals for the other seventeen, all excluding
 269 zero except the residential bot’s two.

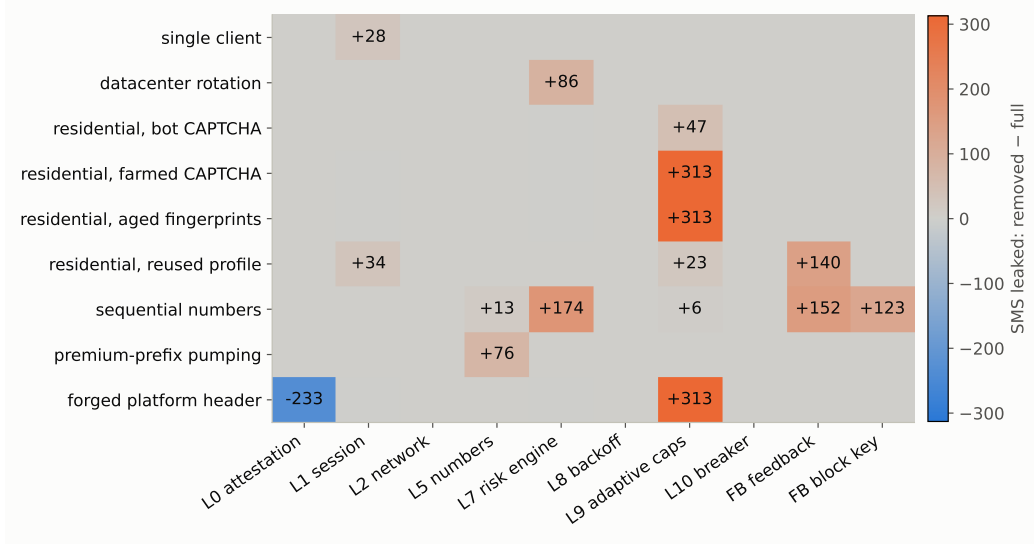


Figure 2: Ablation, caps on: leakage with one layer removed minus the full design, paired on ten seeds and one offered trace; cells within ± 5 unlabelled. L0 to L10 are steps, FB the feedback loop and block key. Without attestation the forged header lowers leakage by moving the attacker into the app bucket.

270 7.3. RQ3: where reputation stops separating

271 A farmed-CAPTCHA attacker uses an address and a fingerprint once and
 272 shares its country, prefix and residential ASN with every legitimate user, so
 273 the penalty its failures produce lands on everyone: rationing, not separation.
 274 Three experiments measure this at the settings run; they do not show that
 275 no setting separates.

276 In Figure 3, up to twice the legitimate rate 99 to 100 % of the attack leaks:
 277 a shared key stays above 0.3 conversion until failures outnumber legitimate
 278 sends about $(0.8 - 0.3)/0.3 \approx 1.7$ to one, and the penalty alone reaches only
 279 *delay*. At five times, 95 % leaks while 6 % of legitimate requests are challenged
 280 and 3 % refused; at ten times, 69 % leaks while 18 % are challenged and 4 %
 281 refused.

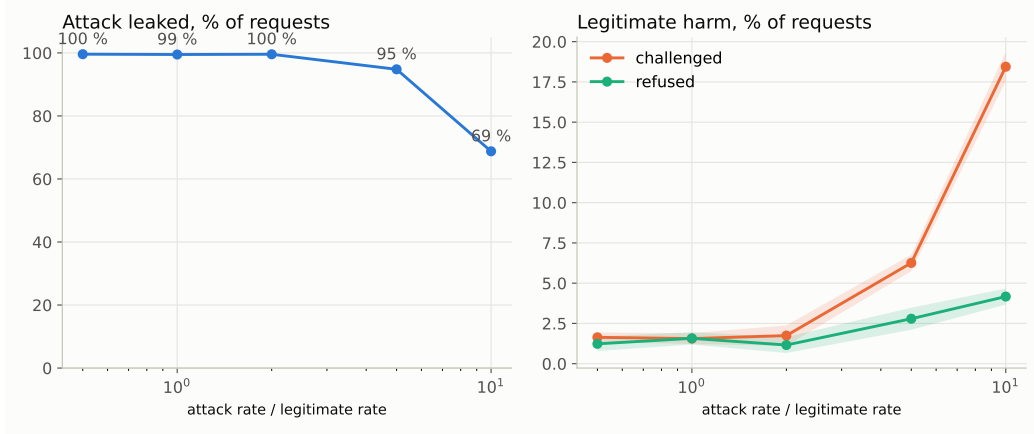


Figure 3: The farm at multiples of the legitimate rate for an hour, caps lifted, five seeds: share of the attack leaked (mean leaked over mean offered), and legitimate requests challenged and refused, with 95 % intervals.

Table 5: One-at-a-time sweep against the farm, caps lifted, ten seeds, all users. Grid: conversion weight 0/10/25/40; fresh-fingerprint weight 0 to 40 by 10; boundary scale 0.6 to 1.4 by 0.2; resolution window 60/120/300/600 s; flood points 0/15/30. Rows not shown are within 10 messages and 5 points of challenge of the default.

Parameter	Value	Leaked	Challenged	Refused
default	—	579 [389, 769]	1.5 %	1.2 %
conversion weight; window	all	579	1.5–1.6 %	1.2–1.3 %
boundary scale	0.6	378 [254, 502]	17.6 %	6.0 %
fresh-fingerprint weight	30	569	6.3 %	2.2 %
fresh-fingerprint weight	40	0	45.8 %	9.6 %

282 In Table 5, the conversion weight and window change nothing: the signal
283 never fires on these keys. Boundaries scaled to 0.6 cut leakage by a third and
284 challenge 18 % of legitimate requests. A fresh-fingerprint weight of 40 stops
285 the attacker by challenging 46 % and refusing 10 %, because most legitimate
286 sign-ups are new browsers too: separation that exists at a price a service must
287 judge. In the settings run, reputation separated an attacker only where its
288 sends dominated the key, the reused profile’s fingerprint and the sequential
289 walk’s block, never a shared key; the key the attacker cannot choose is the
290 destination (Section 7.5).

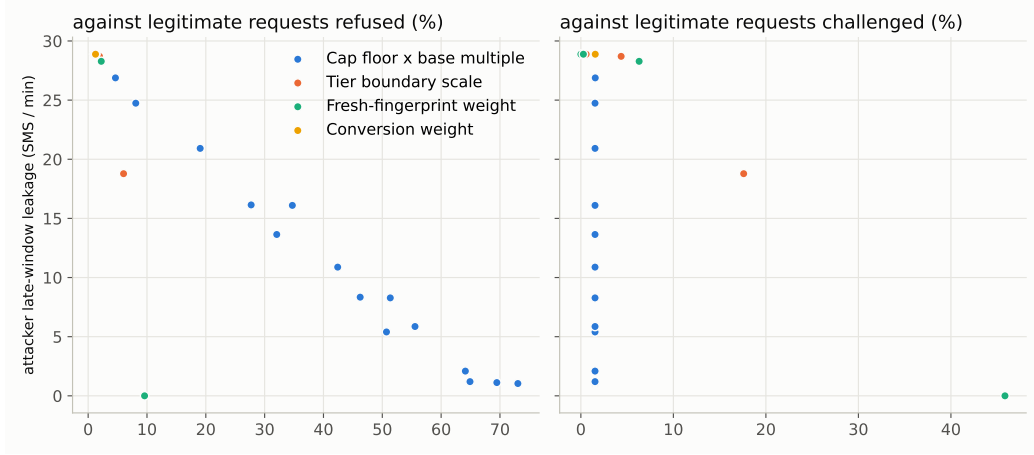


Figure 4: Late-window leakage of the farm against legitimate requests refused (left) and challenged (right), one point per sweep setting, ten seeds; separate axes, so nothing is counted twice. The default is the cap point at 46 % refused and 8.3 messages a minute; intermediate policies were not run.

291 7.4. RQ4: what rationing costs, and what it depends on

292 When separation fails, the cap is the lever (Figure 4). The default (floor
293 0.25, base cap three times the legitimate rate) holds the farm to 8.3 messages
294 a minute and refuses 46 % of legitimate requests; the tightest setting holds it
295 to one a minute and refuses 73 %; no rationing refuses 1 %.

Table 6: Cadence, tick phase and attack length (farm, caps on, ten seeds; values for a tick aligned with the attack start / offset by half a period), and the deployed baseline job against the oracle (five seeds). Share of attack requests leaked; completion and first-time refusal of legitimate requests.

Setting (attack length)	Leaked	Share	Completed	First-time refused
every minute, default (20 min)	233 [206, 261]	40 %	43 %	53 %
every 2 minutes (20 min)	285 / 286	49 / 49 %	50 / 51 %	43 / 42 %
every 5 minutes (20 min)	398 / 546	68 / 94 %	63 / 78 %	26 / 5 %
every 10 minutes (20 min)	521 / 427	89 / 73 %	76 / 68 %	8 / 19 %
hourly (20 min)	546 / 546	94 / 94 %	78 / 78 %	5 / 5 %
hourly (60 min)	1528 / 1628	88 / 94 %	74 / 77 %	9 / 5 %
oracle, five seeds (20 min)	259 [212, 306]	–	42 %	55 %
learned, cold start	649 [346, 953]	–	76 %	7 %
learned, one closed hour	242 [163, 320]	–	36 %	66 %
learned, three closed hours	247 [185, 308]	–	39 %	60 %
learned, three hours poisoned	206 [124, 287]	–	33 %	68 %

296 The gain depends on two things (Table 6). The job accumulates volume
 297 between ticks, so its cadence changes both when and on what it acts: every
 298 two minutes rations nearly as the default does, every five or ten rations or
 299 not depending on the tick’s phase, and hourly leaves the static cap for a
 300 twenty-minute attack and leaks 88 to 94 % of a sixty-minute one. This is
 301 the dependence at these settings, not proof that only a one-minute controller
 302 works. The deployed job, learning from counters that include refused attack
 303 volume, matches the oracle once it has one closed hour and leaves the static
 304 cap with none; three hours poisoned at the legitimate rate are indistinguish-
 305 able from clean ones at five seeds. Days of poisoning, restarts and weekly
 306 drift were not run.

307 7.5. Pumping on the destination, and the detector comparison

Table 7: Concentrated pumpers on three 10 000-number blocks, caps lifted, ten seeds, 130-minute warm-up: leaked of about 560 requests; seeds contained and mean minutes over those seeds; codes the carrier entered; verdict events / distinct blocks. Default: $h = \ln 1000$, $c = 1$, graded verdicts. Legitimate completion is 79 to 80 % in every row.

Carrier, design	Leaked	Contained	Minutes	Codes	Events / blocks
never verifies: v1	556 [369, 744]	0/10	–	0	0 / 0
v2 default, or 24-h denylist	88 [61, 114]	10/10	3.6 [3.1, 4.1]	0	15.8 / 3.0
verifies within 1 s: v1	556	0/10	–	556	0 / 0
v2 default	18 [15, 20]	10/10	1.4 [0.9, 1.9]	18	3.0 / 3.0
verifies 60 % after 30 s: v1	556	0/10	–	336	0 / 0
v2 default ($c = 1$)	445 [345, 546]	0/10	–	268	2.4 / 1.5
v2, $c = 0$ (Page’s CUSUM)	152 [131, 173]	7/10	6.1 [3.6, 8.7]	92	–
v2, unbounded c (SPRT)	556 [369, 744]	0/10	–	336	–
buys challenges: v2 default	101 [76, 126]	10/10	4.1 [3.6, 4.6]	0	18.3 / 3.0
v2, 24-h denylist	88 [61, 114]	10/10	3.6 [3.1, 4.1]	0	15.8 / 3.0
farm (control): v2 default	565 [373, 757]	0/10	–	0	0.1 / 0.1

308 A pumper whose carrier never verifies is contained in every seed, at minute
 309 3.6 on average (3 to 5), at 88 messages (Table 7); one that verifies within a
 310 second is caught by the speed test at 18 and runs at a loss. A pumper buying
 311 challenge solutions leaks 101, against 88 under the hard denylist, which buys
 312 thirteen messages by locking three 10 000-number blocks out for a day. The
 313 carrier verifying 60 % with human-like delay is the hard case: unbounded
 314 credit leaks 556; no credit catches it in seven seeds of ten, at minute 6.1 on
 315 average (4 to 11) in those, at 152; the default does not catch it in the window
 316 and leaks 445, entering 268 codes.

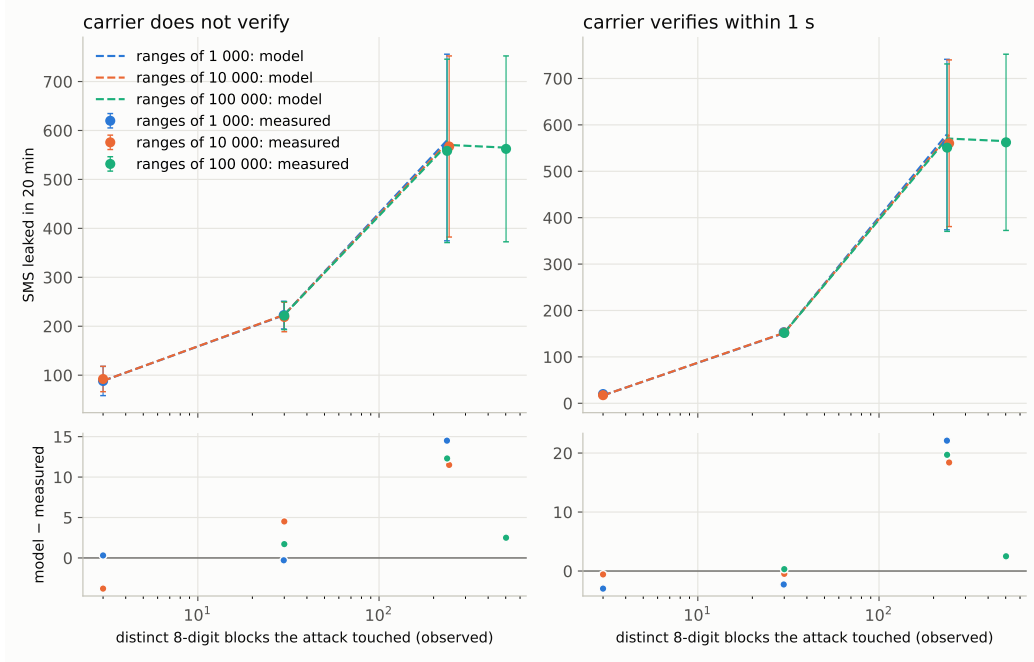


Figure 5: Pumper spread, ten seeds: leakage against distinct blocks touched, for ranges inside, aligned with or spanning the 8-digit key; dashed lines are Equation (2) on the observed blocks, bars 95 % intervals, lower panels the mean per-run residual (model minus measured).

Over 18 spread configurations (Figure 5: two carriers, three range counts, three range sizes), ranges of 1 000 and 10 000 numbers behave indistinguishably, ranges of 100 000 touch ten blocks each, and beyond about 200 blocks the tests do not fire in time. In the ten unsaturated configurations the mean absolute per-run error of Equation (2) is 1 to 9 messages (1 to 15%), at most 21 in one run; in the eight saturated ones it overstates by 2.5 to 22 on average and up to 92 in one run. It checks the k -per-block mechanism, with no per-run guarantee.

Table 8: Detectors on the same seeds. Leaked: human-like (HL) and never-verifying (NV) carriers, ten seeds. Legitimate: 24 hours without attack on 200 distinct blocks at 144 sends per block per day, 65 % conversion, five seeds: verdict events, blocks, requests hit, never completed, completion of about 29 000 requests. At 80 % conversion every sequential row gives at most 2.2 events a day except $c = 0$ at $\ln 1000$ (16.8) and $\ln 100$ (82.4).

Detector	leaked		legitimate, 24 h, 65 %				
	HL	NV	events	blocks	hit	lost	compl.
$h = \ln 1000, c = 1$ (default)	445	88	7.4	7.0	40	17	64.4 %
$h = \ln 1000, c = 0$	152	86	150.6	107.8	778	327	64.2 %
$h = \ln 1000, c = 0.5$	273	88	19.8	19.2	98	45	64.4 %
$h = \ln 1000, c = 2$	556	88	3.8	3.8	22	10	64.4 %
$h = \ln 1000$, unbounded	556	88	3.6	3.6	21	9	64.4 %
$h = \ln 100, c = 0$	120	86	406.8	175.0	2047	895	63.8 %
$h = \ln 100, c = 1$	266	86	38.8	35.2	197	86	64.4 %
$h = \ln 10\,000, c = 0$	266	91	29.2	28.4	148	62	64.4 %
$h = \ln 10\,000, c = 1$	538	91	1.0	1.0	5	1	64.4 %
counter 5/day, refuse	14	14	0	0	0	0	2.3 %
counter 20/day, refuse	58	58	0	0	0	0	9.0 %
counter 5/day, graded	14	14	0	0	26 878	26 628	4.6 %
counter 10/day, graded	29	28	0	0	25 466	24 929	8.8 %
counter 20/day, graded	59	58	0	0	22 939	21 822	16.5 %

325 Table 8 and Figure 6 compare the detectors. Against a carrier that never
326 verifies, every sequential setting leaks 86 to 91: the statistic only climbs.
327 Against the human-like carrier they form a frontier: the settings that leak
328 152 or fewer ($c = 0$ at $\ln 100$ and $\ln 1000$) produce 151 to 407 verdict events
329 a day at 65 % conversion, hitting 778 to 2047 requests; every setting under
330 ten events leaks 445 or more; $c = 0.5$ lies between (273 leaked, 20 events).
331 The default sits at the low-harm end: 7 events, 40 requests hit, 17 never
332 completed. The counter leaks less than every sequential setting against the
333 human-like, never-verifying and challenge-buying carriers (14 to 88, against
334 86 to 556), and against the instant verifier only its five-a-day form matches
335 them (14, against 14 to 24). It completes 2 to 17 % of registrations on blocks
336 receiving 144 sends a day at 65 % conversion (3 to 21 % at 80 %), whichever
337 action it takes, against 64 % (79 %) for the sequential settings. Sparser blocks,
338 matched in-control run lengths and windowed alternatives were not run.

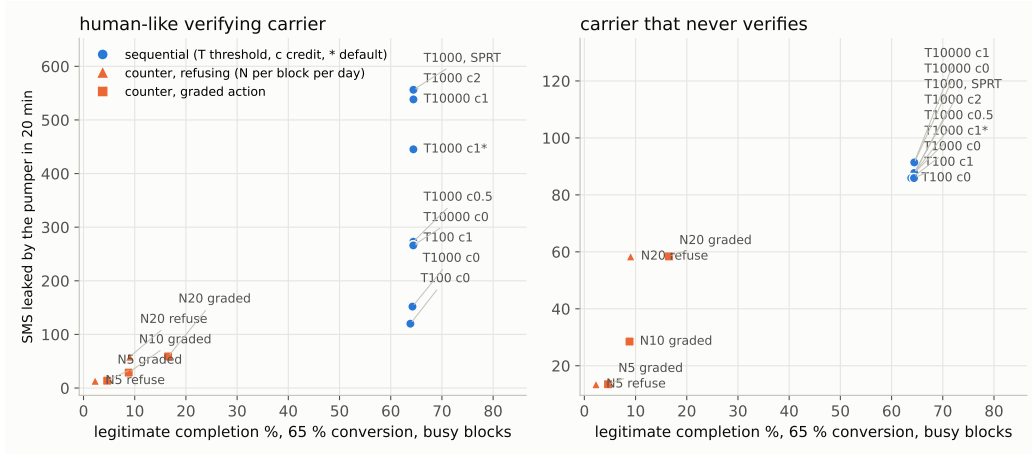


Figure 6: Table 8 as points: leakage of the human-like (left) and never-verifying (right) carrier against legitimate completion at 65 % conversion on busy blocks. Sequential settings share one completion because verdicts rarely refuse.

339 7.6. RQ5: what the block tests cost legitimate users

Table 9: Probability that a block with only legitimate traffic reaches a verdict over 30 resolved sends from empty statistics, by true conversion and share of verifications within 5s; tests as deployed. Events of 20000 trials with 95 % Wilson intervals; zero bounds the probability below about 0.02 %, not at zero.

Conversion	fast 0 %	fast 20 %	fast 30 %
50 %	3933: 19.7 % [19.1, 20.2]	3895: 19.5 % [18.9, 20.0]	4068: 20.3 % [19.8, 20.9]
65 %	377: 1.88 % [1.71, 2.08]	378: 1.89 % [1.71, 2.09]	522: 2.61 % [2.40, 2.84]
80 %	14: 0.07 % [0.04, 0.12]	18: 0.09 % [0.06, 0.14]	139: 0.69 % [0.59, 0.82]
90 %	0: 0.00 % [0.00, 0.02]	12: 0.06 % [0.03, 0.10]	158: 0.79 % [0.68, 0.92]

340 First, a Monte Carlo of the deployed tests on one block from empty
 341 statistics (Table 9) gives one block's hazard per window, not the daily harm.
 342 Assuming 80 % conversion and 20 % fast verifications, a population matching
 343 those assumptions triggers a verdict in about one window in a thousand at 30
 344 sends, 0.7 % if autofill is 30 %; at 65 % conversion about 2 % at 30 sends and
 345 2.5 to 3.5 % at 100; at 50 %, 20 % at 30 sends and 41 % at 100. Conversion
 346 is the sensitive assumption and must be measured.

347 Second, 24 hours of legitimate traffic only, five seeds per cell, on 200 or
 348 1000 distinct blocks (144 or 29 sends a day; observed maxima 181 and 49) or
 349 spread over all prefixes (about 4), with the speed calibration fixed or recal-

350 brated. At 80 % conversion and 20 % autofill the day gives 0.2 to 2.2 verdict
 351 events and at most 1.4 requests hit of about 28 800; 30 % autofill against the
 352 fixed calibration gives up to 7.6 events and 7 hits, and recalibration at most
 353 1.6 events. At 65 % conversion either protocol gives 7 to 27 events, hitting
 354 38 to 42 requests at 144 sends a day (16 to 18 never completed), 22 to 29 at
 355 29 and 4 at 4. Five seeds bound a mean, not rare cascades.

356 Third, one prefix, a seventh of the traffic, stops for thirty minutes, eight
 357 seeds. With receipts and the suspension, an outage reported as failures gives
 358 0.1 [0, 0.4] verdict events and a silent one 0.9 [0, 1.7]; clocking from the send
 359 gives 1.9 [0.7, 3.0] in both. Receipts reduce the effect, they do not remove it.

Table 10: The block poisoner, caps lifted, ten seeds: verdict events (stage 1 / 2), blocks, legitimate requests hit (stage 1 / 2), hit and never completed (lost), hits after the attacker stopped (attack for 10 of 30 minutes), completion.

Variant	Events	Blocks	Hit	Lost	After stop	Compl.
graded (default)	18.8 (13.9 / 4.9)	13.9	100 (69 / 31)	49 [16, 82]	–	71 %
hard deny, 24 h	20.1	13.9	119	119 [52, 185]	–	54 %
graded, stops at 10 min	12.7 (9.1 / 3.6)	9.1	166 (114 / 52)	84	150 [60, 241]	70 %

360 Fourth, the block poisoner (Table 10) earns 19 verdict events on 14 blocks,
 361 a quarter of them second-stage, and hits 100 legitimate requests, 49 of which
 362 never complete; the hard denylist loses all 119 it hits. After the attacker
 363 stops, its verdicts hit 150 more requests in twenty minutes. Grading limits
 364 the harm but caps it neither at a challenge nor at the attack’s duration. This
 365 is the central risk of destination-keyed reputation.

366 7.7. Adaptive attackers, economics and overhead

367 A carrier on a listed elevated-cost range is held to 7 messages when it
 368 verifies instantly and 20 with human-like delay; spread over all prefixes it is
 369 a verifying flooder, untouched at 579. A datacenter attacker buying every
 370 challenge solution leaks 54 to 57. Low-and-slow attacks leak at their own
 371 rate: 40 at two requests a minute, 188 to 194 at ten. The trust builder’s
 372 500 pairs leak 264 while verifying for ten minutes and then 250 of 281 flood
 373 requests (162 and 82 with caps on); concentrated in three blocks, its flood
 374 leaks 164, because its blocks earn 28 verdict events. Bought trust is not
 375 told from earned trust; the floor only bounds how much a block banks. The
 376 receipt faker leaks 575 (237 with caps on) with no verdict. Only caps and
 377 budget bound the receipt faker and the spread trust builder.

378 In scenario accounting that credits pumpers with 20 to 50 % of the retail
379 price, premium pumping earns 14 to 40 USD per twenty minutes under v1
380 and nothing under v2; under v2 the concentrated pumper loses up to 1.19
381 USD when verifying instantly, earns up to 4.55 USD when never verifying
382 and 11 to 30 USD when verifying like a person.

383 On a real Redis with four vCPUs and four workers, a send costs 12.6 ms
384 at the median and 19.2 ms at the 99th percentile, with 48 round trips. With
385 each fake vendor call taking a lognormal 50 ms and the floor on, throughput
386 is 68, 115 and 131 requests per second at concurrency 32, 64 and 128; at 128,
387 25 % of requests exceed the floor and leak timing, while 99.9 % of happy-
388 path requests are still sent. With the floor, no pair of five outcome classes
389 was distinguished by a Kolmogorov–Smirnov test (smallest $p = 0.10$) and all
390 means were within 2 ms by two one-sided tests; without it nine of ten pairs
391 were distinguished. That is a failure to detect a difference, not proof that
392 timing carries no information.

393 8. Discussion

394 *Lessons.* In the settings run, reputation separated attackers only on keys
395 they dominated; a five-a-day counter was the strongest destination limiter
396 and the costliest on busy blocks; the detector’s memory has no free point in
397 the grid; verdicts can be aimed at legitimate users; and receipts come from
398 the adversary’s side of the network. Rationing needs a fast controller with
399 history, and block tests need measured calibration.

400 *Limitations.* Everything is simulated, and the calibration is assumed; inter-
401 vals cover seeds, not those choices. Several results follow from the generator
402 (premium numbers in a perfect list, a datacenter attacker that declines chal-
403 lenges, a farm with human scores), and profiles and defaults were refined
404 against the same generator, a selection risk we did not control with held-out
405 scenarios. Legitimate users make one request each, with no retries or bursts;
406 multi-day attacks and correlated vendor failures are absent; fallback (70 %)
407 and challenge completion (90 %) were not swept. The main study’s base-
408 line is an oracle; the detector comparison covers one block density without
409 matched run lengths, windowed, time-uniform, puzzle or supervised alterna-
410 tives. About 1 % of the concentrated trust builder’s pairs share a number.
411 The real-Redis tests establish only the five properties of Section 6; vendor
412 adapters are untested live; the timing test excludes vendor latency and 403
413 responses.

414 *Ethics and privacy.* The design keeps fingerprint first-seen times for 90 days
415 and audit records for thirty, and challenges people for what strangers sharing
416 their network or number block did; a deployment under the GDPR or Saudi
417 Arabia’s PDPL needs a lawful basis, a retention statement and a route for
418 refused users. No human data was used. The incident is described without
419 identifying the operator.

420 **9. Conclusion**

421 In this simulation the design contains attacks that carry a signature, with
422 completion within five points of the no-attack rate, and contains a pumper
423 concentrated on a few destination blocks at a cost a closed-form estimate
424 reproduces. It does not separate a residential attacker that looks like a person
425 at a cost a service would accept; it rations, and rationing works only with a
426 fast controller that has history. Carriers that verify their own codes, build
427 trust or fake receipts defeat the destination tests, and attackers can aim the
428 tests at legitimate users. The detector’s memory is a measured frontier with a
429 five-a-day counter at its extreme: fewest messages leaked, most registrations
430 lost. Replaying real traffic would show how much of this survives.

431 **Artifact availability and supplementary material**

432 Code, simulation, results and scripts are at [https://github.com](https://github.com/nahmedpsu/OTP-DDOS)
433 [/nahmedpsu/OTP-DDOS](https://github.com/nahmedpsu/OTP-DDOS), release 2.6.0 (a DOI will be added on ac-
434 ceptance). `make results` regenerates the evaluation from fixed seeds,
435 `paper/figures.py` draws every figure from `results/evaluation.json`,
436 and `results/headline_numbers.md` maps headline numbers to their
437 study, seeds and JSON path. The complete tables with intervals
438 (`results/evaluation.md`, `results/performance.md`) are the supplemen-
439 tary material.

440 **CRedit authorship contribution statement**

441 **Naveed Ahmed:** Conceptualization, Methodology, Software, Valida-
442 tion, Investigation, Writing – original draft, Writing – review & editing.

443 **Declaration of competing interest**

444 The author declares no known competing financial interests or personal
445 relationships that could have appeared to influence this work.

446 Declaration of generative AI and AI-assisted technologies in the 447 writing process

448 The author used Claude (Anthropic) to assist with code, the evaluation,
449 and drafting and editing the text, checked the numbers against the recorded
450 results and the claims against the code, and takes full responsibility for the
451 content.

452 Data availability

453 No real traffic data was used; all synthetic data, seeds and results are in
454 the repository.

455 References

- 456 [1] MITRE Corporation, Resource hijacking: SMS pumping, technique
457 T1496.003, MITRE ATT&CK, <https://attack.mitre.org/techniques/T1496/003/>, accessed 1 October 2026.
- 459 [2] Commsrisk, Elon Musk says Twitter lost \$60mn a year because 390
460 telcos used bot accounts to pump A2P SMS, <https://commsrisk.com/elon-musk-says-twitter-lost-60mn-a-year-because-390-telcos-used-bot-accounts-to-pump-a2p-sms/>, trade press report of a
462 public statement; the figure is the company’s own claim (2023).
- 464 [3] J. H. Huh, H. Shin, S. Ahn, H. Yi, J. Cho, T. Kim, M. Lim, N. Choi,
465 Preventing artificially inflated SMS attacks through large-scale traffic in-
466 spection, in: 34th USENIX Security Symposium, USENIX Association,
467 Seattle, WA, 2025.
- 468 [4] M. Sahin, A. Francillon, P. Gupta, M. Ahamad, SoK: Fraud in telephony
469 networks, in: 2017 IEEE European Symposium on Security and Privacy
470 (EuroS&P), IEEE, 2017. doi:10.1109/EuroSP.2017.40.
- 471 [5] M. Sahin, A. Francillon, Understanding and detecting international rev-
472 enue share fraud, in: Network and Distributed System Security Sympo-
473 sium (NDSS), Internet Society, 2021. doi:10.14722/ndss.2021.24051.
- 474 [6] Twilio Inc., Verify fraud guard, [https://www.twilio.com/docs/veri-
475 fy/preventing-toll-fraud/sms-fraud-guard](https://www.twilio.com/docs/verify/preventing-toll-fraud/sms-fraud-guard), product documenta-
476 tion. Accessed 1 October 2026.

- 477 [7] Twilio Inc., SMS pumping risk score (Lookup v2), <https://www.twilio.com/docs/lookup/v2-api/sms-pumping-risk>, product documenta-
478 tion. Accessed 1 October 2026.
479
- 480 [8] M. Motoyama, K. Levchenko, C. Kanich, D. McCoy, G. M. Voelker,
481 S. Savage, Re: CAPTCHAs – understanding CAPTCHA-solving ser-
482 vices in an economic context, in: 19th USENIX Security Symposium,
483 USENIX Association, 2010.
- 484 [9] A. Dionysiou, E. Athanasopoulos, SoK: Machine vs. machine – A sys-
485 tematic classification of automated machine learning-based CAPTCHA
486 solvers, *Computers & Security* 97 (2020) 101947. doi:10.1016/j.cose
487 .2020.101947.
- 488 [10] D. Dean, A. Stubblefield, Using client puzzles to protect TLS, in: 10th
489 USENIX Security Symposium, USENIX Association, 2001.
- 490 [11] X. Mi, X. Feng, X. Liao, B. Liu, X. Wang, F. Qian, Z. Li, S. Alrizah,
491 L. Chen, Y. Qian, Resident evil: Understanding residential IP proxy
492 as a dark service, in: 2019 IEEE Symposium on Security and Privacy
493 (S&P), IEEE, 2019. doi:10.1109/SP.2019.00011.
- 494 [12] P. Richter, F. Wolsing, A. Chandrasekaran, R. Padmanabhan, M. All-
495 man, R. Bush, A. Feldmann, A multi-perspective analysis of carrier-
496 grade NAT deployment, in: *Proceedings of the 2016 ACM Internet
497 Measurement Conference (IMC)*, ACM, 2016. doi:10.1145/298744
498 3.2987474.
- 499 [13] J. R. Douceur, The Sybil attack, in: *Peer-to-Peer Systems (IPTPS
500 2002)*, Vol. 2429 of LNCS, Springer, 2002, pp. 251–260. doi:10.100
501 7/3-540-45748-8_24.
- 502 [14] A. Wald, Sequential tests of statistical hypotheses, *The Annals of Math-
503 ematical Statistics* 16 (2) (1945) 117–186. doi:10.1214/aoms/1177731
504 118.
- 505 [15] E. S. Page, Continuous inspection schemes, *Biometrika* 41 (1/2) (1954)
506 100–115. doi:10.2307/2333009.

- 507 [16] S. R. Howard, A. Ramdas, J. McAuliffe, J. Sekhon, Time-uniform, non-
508 parametric, nonasymptotic confidence sequences, *The Annals of Statis-*
509 *tics* 49 (2) (2021) 1055–1080. doi:10.1214/20-AOS1991.
- 510 [17] S. Wiefling, L. Lo Iacono, M. Dürmuth, Is this really you? An empirical
511 study on risk-based authentication applied in the wild, in: *ICT Systems*
512 *Security and Privacy Protection (IFIP SEC 2019)*, Vol. 562 of *IFIP Ad-*
513 *vances in Information and Communication Technology*, Springer, 2019,
514 pp. 134–148. doi:10.1007/978-3-030-22312-0_10.
- 515 [18] C. Mulliner, R. Borgaonkar, P. Stewin, J.-P. Seifert, SMS-based one-
516 time passwords: Attacks and defense, in: *Detection of Intrusions and*
517 *Malware, and Vulnerability Assessment (DIMVA 2013)*, Vol. 7967 of
518 *LNCS*, Springer, 2013, pp. 150–159. doi:10.1007/978-3-642-39235
519 -1_9.
- 520 [19] H. Siadati, T. Nguyen, P. Gupta, M. Jakobsson, N. Memon, Mind your
521 SMSes: Mitigating social engineering in second factor authentication,
522 *Computers & Security* 65 (2017) 14–28. doi:10.1016/j.cose.2016.
523 09.009.
- 524 [20] N. Ahmed, OTP Guard: a layered SMS validation, risk-scoring and
525 rate-limiting pipeline against OTP flood abuse (version 2.6.0), <https://github.com/nahmedpsu/OTP-DDOS>, source code, evaluation and
526 recorded results. Archived DOI to be added (2026).
527
- 528 [21] Google LLC, SMS retriever API overview, <https://developers.google.com/identity/sms-retriever/overview>, developer documenta-
529 tion. Accessed 1 October 2026.
530
- 531 [22] Web Incubator Community Group, WebOTP API, <https://github.com/WICG/WebOTP>, specification repository. Accessed 1 October 2026.
532
- 533 [23] Twilio Inc., Verify API, <https://www.twilio.com/en-us/user-authentication-identity/verify>, product page stating a 68%+ global
534 conversion rate. Accessed 1 October 2026.
535